COURS POUR DÉBUTANTS · ANDROID

Les thèmes en Jetpack Compose

Couleurs, typographie, formes, mode sombre
avec Material 3 — exemples et captures d'écran

Ce que vous allez apprendre

- Comprendre ce qu'est un thème et à quoi sert `MaterialTheme`.
- Définir vos propres couleurs avec `ColorScheme` (rôles `primary`, `onPrimary`…).
- Gérer automatiquement le mode clair et le mode sombre.
- Personnaliser la typographie et les formes.
- Utiliser le thème dans vos composables, sans jamais coder une couleur « en dur ».
- Aller plus loin : couleurs dynamiques, couleurs personnalisées, thèmes imbriqués.

Prérequis : savoir créer un projet Android Studio « Empty Activity » et écrire une fonction `@Composable` simple (`Text`, `Column`, `Button`).

Fil rouge : nous construisons pas à pas le thème d'une petite application « Mon Jardin » qui liste des plantes à arroser. Le code complet est fourni dans le dossier `code-android/` de l'archive.

Sommaire

1. Qu'est-ce qu'un thème ? 3

2. Les fichiers du thème dans un projet 5

3. Les couleurs : ColorScheme 6

3.1 La notion de rôle . . . 6

3.2 Déclarer ses couleurs (Color.kt) . . . 6

4. Assembler le thème (Theme.kt) 9

5. Mode clair, mode sombre et couleurs dynamiques 10

5.1 Le mode sombre automatique . . . 10

5.2 Les couleurs dynamiques (Material You) . . . 10

5.3 Forcer un mode (réglage dans l'application) . . . 11

6. La typographie : Typography 12

6.1 Utiliser une police personnalisée . . . 13

6.2 Utiliser un style dans un écran . . . 13

7. Les formes : Shapes 14

8. Utiliser le thème dans ses composables 15

8.1 Les composants Material s'adaptent tout seuls . . . 15

8.2 Lire une valeur du thème . . . 16

8.3 Surface et la couleur de contenu automatique . . . 16

8.4 L'erreur classique : la couleur codée en dur . . . 17

9. Aller plus loin 19

9.1 Ajouter ses propres couleurs (CompositionLocal) . . . 19

9.2 Thème imbriqué : changer le style d'une seule zone . . . 20

10. Bonnes pratiques et mémo 22

10.1 Bonnes pratiques . . . 22

10.2 Mémo . . . 22

10.3 Exercices . . . 22

10.4 Contenu de l'archive . . . 23

1. Qu'est-ce qu'un thème ?

Dans une application, un **thème** est un ensemble de choix visuels partagés par tous les écrans : les couleurs, les polices de caractères et les formes (arrondis des boutons, des cartes...). Au lieu de régler la couleur de chaque bouton un par un, on décrit le style **une seule fois** et tous les composants s'en servent automatiquement.

En Jetpack Compose, le thème est fourni par le composable `MaterialTheme` (bibliothèque **Material 3**). Il contient trois « piliers » :

Pilier	Classe	Ce qu'il définit
Couleurs	`ColorScheme`	Les couleurs de l'application, organisées par rôles (primary, surface, error...)
Typographie	`Typography`	Les styles de texte : titres, corps de texte, libellés...
Formes	`Shapes`	Les arrondis ou coins coupés des cartes, boutons, boîtes de dialogue...

Tout composable placé à l'intérieur de `MaterialTheme { … }` peut lire ces valeurs grâce à trois propriétés :

```
MaterialTheme.colorScheme   // les couleurs
MaterialTheme.typography    // les styles de texte
MaterialTheme.shapes        // les formes
```

Voici la même interface affichée avec le thème par défaut de Material 3 (violet) puis avec notre thème « Jardin » (vert). **Le code de l'écran est strictement identique** : seul le thème qui l'entoure change.



Thème Material 3 par défaut



Thème « Jardin » personnalisé

À retenir
Un thème est un « réservoir de style » placé en haut de l'arborescence de composables. Les composants Material (Button, Card, TopAppBar…) vont y chercher leurs couleurs, polices et formes.

2. Les fichiers du thème dans un projet

Quand vous créez un projet « Empty Activity » dans Android Studio, un dossier `ui/theme` est généré automatiquement :

```
app/src/main/java/com/exemple/monjardin/
├── MainActivity.kt
└── ui/
    └── theme/
        ├── Color.kt    ← les couleurs
        ├── Type.kt     ← la typographie
        ├── Shape.kt    ← les formes (à créer si absent)
        └── Theme.kt    ← la fonction qui assemble le tout
```

Le fichier `Theme.kt` contient une fonction composable (par exemple `MonJardinTheme`, nommée d'après votre projet) qui appelle `MaterialTheme`. Dans `MainActivity`, tout l'écran est enveloppé dans cette fonction :

MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            JardinTheme {          // Tout ce qui est à l'intérieur hérite du thème
                EcranJardin()
            }
        }
    }
}
```

> **Astuce**
> Enveloppez aussi vos fonctions `@Preview` dans votre thème, sinon l'aperçu d'Android Studio utilisera les couleurs par défaut et ne ressemblera pas à votre application.

Aperçus en mode clair et sombre

```
@Preview(name = "Clair", showBackground = true)
@Composable
fun ApercuClair() { JardinTheme(darkTheme = false) { EcranJardin() } }

@Preview(name = "Sombre", showBackground = true)
@Composable
fun ApercuSombre() { JardinTheme(darkTheme = true) { EcranJardin() } }
```

3. Les couleurs : ColorScheme

3.1 La notion de rôle

Material 3 ne vous demande pas « la couleur du bouton » mais des couleurs par **rôle**. Chaque rôle a un usage précis, et presque chaque couleur a son partenaire **on…** : la couleur du texte ou de l'icône qui s'affiche par-dessus.

Rôle	Utilisation typique
`primary / onPrimary`	Éléments principaux : bouton plein, bouton flottant (FAB), éléments sélectionnés.
`primaryContainer / onPrimaryContainer`	Zones mises en avant mais plus douces : barre du haut, FAB, puces.
`secondary / secondaryContainer`	Éléments secondaires : bouton « tonal », indicateur de la barre de navigation.
`tertiary / tertiaryContainer`	Couleur d'accent pour contraster (badges, zone spéciale).
`error / errorContainer`	Messages et champs en erreur.
`background, surface / onSurface`	Fond de l'écran, des cartes, des menus ; couleur du texte par défaut.
`surfaceVariant / onSurfaceVariant`	Fonds alternatifs, texte secondaire (moins important).
`outline`	Bordures : champ de texte, bouton « outlined ».

> **À retenir**
> Règle d'or : un texte posé sur `X` utilise la couleur `onX`. Exemple : du texte sur `primary` → `onPrimary`. C'est ce qui garantit un bon contraste.

3.2 Déclarer ses couleurs (Color.kt)

On crée deux palettes, une claire avec `lightColorScheme()` et une sombre avec `darkColorScheme()`. Les couleurs s'écrivent au format `0xAARRGGBB` (AA = opacité, FF = opaque). Les rôles non précisés prennent une valeur par défaut.

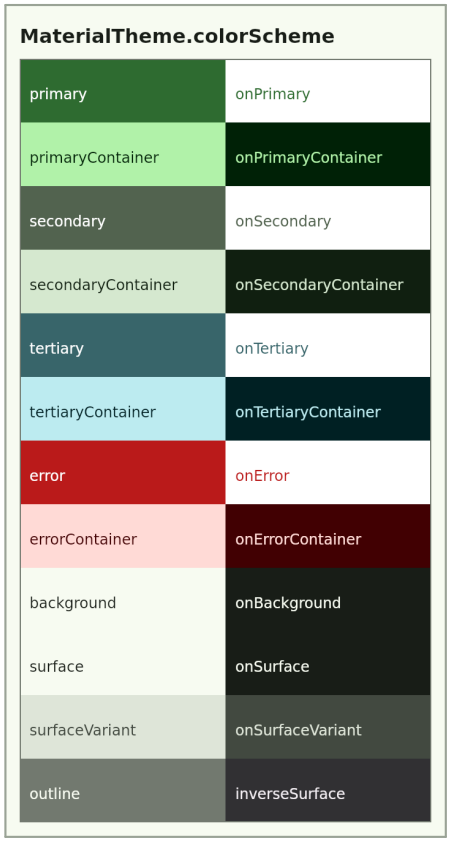
```
ui/theme/Color.kt
```

```
val Vert40 = Color(0xFF2E6B30)
val Vert80 = Color(0xFF96D68F)
val VertClair = Color(0xFFB1F2A9)

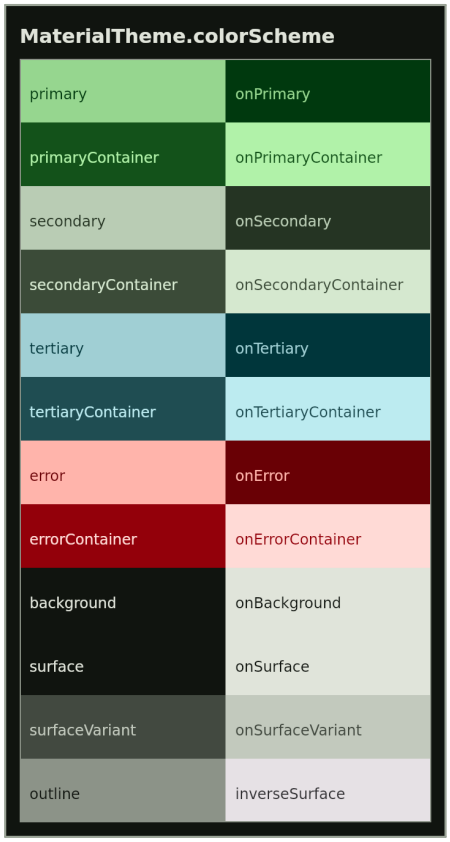
val LightColors = lightColorScheme(
    primary = Vert40, onPrimary = Color.White,
    primaryContainer = VertClair, onPrimaryContainer = Color(0xFF002106),
    secondary = Color(0xFF52634F), onSecondary = Color.White,
    secondaryContainer = Color(0xFFD5E8CF), onSecondaryContainer = Color(0xFF101F10),
    tertiary = Color(0xFF38656A), onTertiary = Color.White,
    tertiaryContainer = Color(0xFFBCEBF0), onTertiaryContainer = Color(0xFF002023),
    error = Color(0xFFBA1A1A), onError = Color.White,
    background = Color(0xFFF7FBF1), onBackground = Color(0xFF181D17),
    surface = Color(0xFFF7FBF1), onSurface = Color(0xFF181D17),
    surfaceVariant = Color(0xFFDEE5D8), onSurfaceVariant = Color(0xFF424940),
    outline = Color(0xFF72796F),
)

val DarkColors = darkColorScheme(
    primary = Vert80, onPrimary = Color(0xFF00390E),
    primaryContainer = Color(0xFF13521A), onPrimaryContainer = VertClair,
    secondary = Color(0xFFB9CCB4), onSecondary = Color(0xFF253423),
    // ... même principe pour les autres rôles (voir code-android/)
    background = Color(0xFF10140F), onBackground = Color(0xFFE0E4DA),
    surface = Color(0xFF10140F), onSurface = Color(0xFFE0E4DA),
)
```

Voici les deux palettes obtenues. Remarquez qu'en mode sombre, les couleurs principales deviennent plus **claires** (vert pastel) et les fonds deviennent foncés : c'est normal, cela conserve la lisibilité.



Palette claire (LightColors)



Palette sombre (DarkColors)

> **Astuce**
> Inutile de choisir 25 couleurs à la main ! L'outil en ligne **Material Theme Builder** (material-foundation.github.io/material-theme-builder) génère toute la palette claire et sombre à partir d'**une seule** couleur de départ, et exporte directement les fichiers `Color.kt` et `Theme.kt`.

4. Assembler le thème (Theme.kt)

La fonction de thème choisit la bonne palette puis appelle `MaterialTheme`. Le paramètre `content` représente tous les composables qui seront « à l'intérieur » du thème.

ui/theme/Theme.kt

```
@Composable
fun JardinTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = false, // true = couleurs du fond d'écran (Android 12+)
    content: @Composable () -> Unit,
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context)
            else dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColors
        else -> LightColors
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = JardinTypography,   // voir chapitre 6
        shapes = JardinShapes,           // voir chapitre 7
        content = content,
    )
}
```

Ligne par ligne :

- `isSystemInDarkTheme()` renvoie `true` si le téléphone est réglé en mode sombre. C'est la valeur par défaut du paramètre.
- `dynamicColor` active les **couleurs dynamiques** (voir chapitre 5).
- Le bloc `when` choisit la palette : dynamique, sombre ou claire.
- `MaterialTheme(...)` rend ces valeurs disponibles à tous les composables de `content`.

5. Mode clair, mode sombre et couleurs dynamiques

5.1 Le mode sombre automatique

Grâce à `isSystemInDarkTheme()`, l'application suit automatiquement le réglage du téléphone. Il n'y a **rien d'autre à faire** dans vos écrans, à condition d'utiliser les couleurs du thème.



JardinTheme(darkTheme = false)



JardinTheme(darkTheme = true)

5.2 Les couleurs dynamiques (Material You)

Depuis Android 12 (API 31), le système peut générer une palette à partir du **fond d'écran** de l'utilisateur. Les fonctions `dynamicLightColorScheme(context)` et `dynamicDarkColorScheme(context)` renvoient alors un `ColorScheme` complet. Sur les versions plus anciennes, elles n'existent pas : c'est pourquoi on teste `Build.VERSION.SDK_INT >= Build.VERSION_CODES.S` et qu'on garde une palette de secours.

Attention
Avec les couleurs dynamiques, **votre** palette n'est plus utilisée sur Android 12+ : l'application prend les couleurs de l'utilisateur. Si l'identité de votre marque est importante (logo vert, par exemple), laissez `dynamicColor = false`.

5.3 Forcer un mode (réglage dans l'application)

Pour proposer un choix « Clair / Sombre / Système » dans les réglages, il suffit de passer une valeur à `darkTheme` :

```
enum class ModeTheme { CLAIR, SOMBRE, SYSTEME }

@Composable
fun App() {
    var mode by rememberSaveable { mutableStateOf(ModeTheme.SYSTEME) }
    val sombre = when (mode) {
        ModeTheme.CLAIR -> false
        ModeTheme.SOMBRE -> true
        ModeTheme.SYSTEME -> isSystemInDarkTheme()
    }
    JardinTheme(darkTheme = sombre) {
        EcranJardin()
    }
}
```

Astuce
Pour que le choix survive à la fermeture de l'application, enregistrez-le avec **DataStore** plutôt qu'avec `rememberSaveable`.

6. La typographie : Typography

Material 3 définit une **échelle typographique** de 15 styles répartis en 5 familles (Display, Headline, Title, Body, Label), chacune en 3 tailles (Large, Medium, Small).

Famille	Usage
`display…`	Très grands textes, rares (chiffre clé, écran d'accueil).
`headline…`	Titres de page.
`title…`	Titres de section, de carte, de barre d'application.
`body…`	Texte courant (paragraphes, descriptions).
`label…`	Petits textes de composants : boutons, onglets, puces.

On crée un objet `Typography` en ne redéfinissant que les styles qui nous intéressent ; les autres gardent leur valeur par défaut :

ui/theme/Type.kt

```
val Titres = FontFamily.Serif
val Corps = FontFamily.SansSerif

val JardinTypography = Typography(
    displayLarge = TextStyle(fontFamily = Titres, fontWeight = FontWeight.Bold,
                             fontSize = 48.sp, lineHeight = 56.sp),
    headlineMedium = TextStyle(fontFamily = Titres, fontWeight = FontWeight.Bold,
                               fontSize = 28.sp, lineHeight = 36.sp),
    titleLarge = TextStyle(fontFamily = Titres, fontWeight = FontWeight.SemiBold,
                           fontSize = 22.sp, lineHeight = 28.sp),
    titleMedium = TextStyle(fontFamily = Corps, fontWeight = FontWeight.SemiBold,
                            fontSize = 16.sp, lineHeight = 24.sp),
    bodyLarge = TextStyle(fontFamily = Corps, fontSize = 16.sp, lineHeight = 24.sp),
    labelLarge = TextStyle(fontFamily = Corps, fontWeight = FontWeight.Bold,
                           fontSize = 14.sp, letterSpacing = 0.5.sp),
)
```



Typography() par défaut



JardinTypography (titres en Serif, gras)

6.1 Utiliser une police personnalisée

1. Copiez votre fichier de police (ex. `lora_bold.ttf`, nom en minuscules, sans tiret) dans `res/font/`.
2. Déclarez la famille et utilisez-la dans vos styles :

```
val Lora = FontFamily(
    Font(R.font.lora_regular, FontWeight.Normal),
    Font(R.font.lora_bold, FontWeight.Bold),
)
// puis : headlineMedium = TextStyle(fontFamily = Lora, ...)
```

6.2 Utiliser un style dans un écran

```
Text("Bonjour !", style = MaterialTheme.typography.headlineMedium)
Text("3 plantes ont besoin de vous.", style = MaterialTheme.typography.bodyMedium)
```

Attention
Évitez `Text("…", fontSize = 28.sp)` éparpillé dans vos écrans. Si vous changez d'avis sur la taille des titres, il faudrait modifier chaque écran. Avec le thème, une seule ligne suffit.

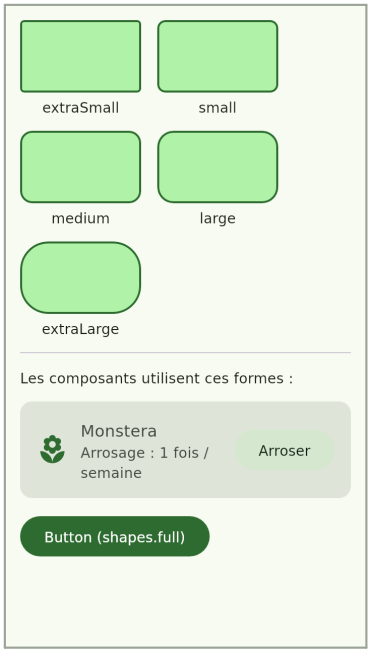
7. Les formes : Shapes

L'objet `Shapes` propose 5 tailles : `extraSmall`, `small`, `medium`, `large`, `extraLarge`. Chaque composant Material utilise l'une d'elles : par exemple les cartes utilisent `medium`, les boîtes de dialogue `extraLarge`, les champs de texte `extraSmall`.

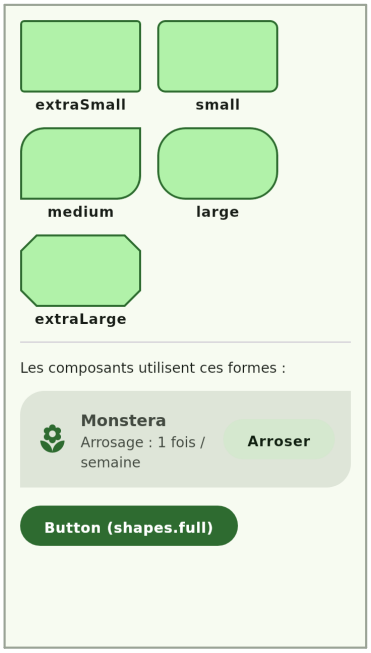
ui/theme/Shape.kt

```
val JardinShapes = Shapes(
    extraSmall = RoundedCornerShape(4.dp),
    small = RoundedCornerShape(8.dp),
    medium = RoundedCornerShape(topStart = 24.dp, bottomEnd = 24.dp), // forme « feuille »
    large = RoundedCornerShape(28.dp),
    extraLarge = CutCornerShape(16.dp),                               // coins coupés
)
```

À gauche les formes par défaut, à droite les nôtres. La carte de plante a pris automatiquement la forme « feuille » (`medium`) sans qu'on modifie son code :



Shapes() par défaut



JardinShapes

Vous pouvez aussi réutiliser une forme du thème sur vos propres éléments :

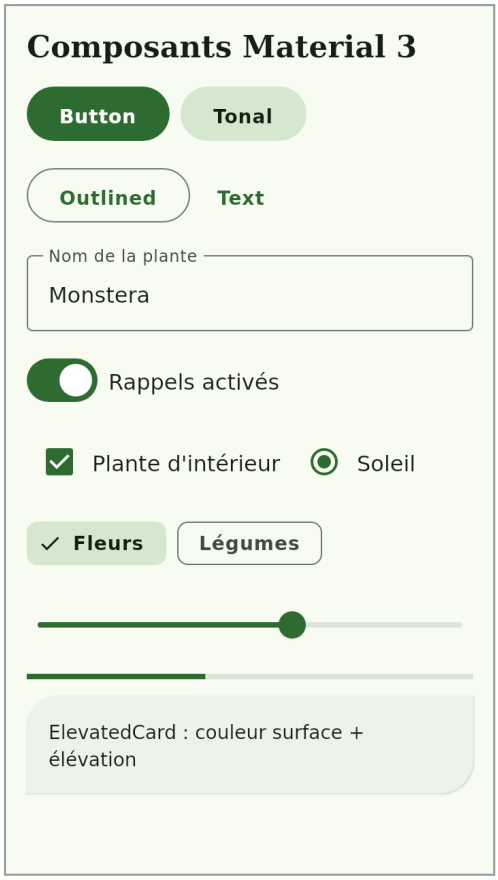
```
Box(
    Modifier
        .size(120.dp)
        .clip(MaterialTheme.shapes.large)
        .background(MaterialTheme.colorScheme.primaryContainer)
)
```

> **À retenir**
> Les boutons (`Button`) ne suivent pas `shapes` : ils sont toujours en forme de « pilule ». Pour les changer, passez `shape = …` directement au bouton.

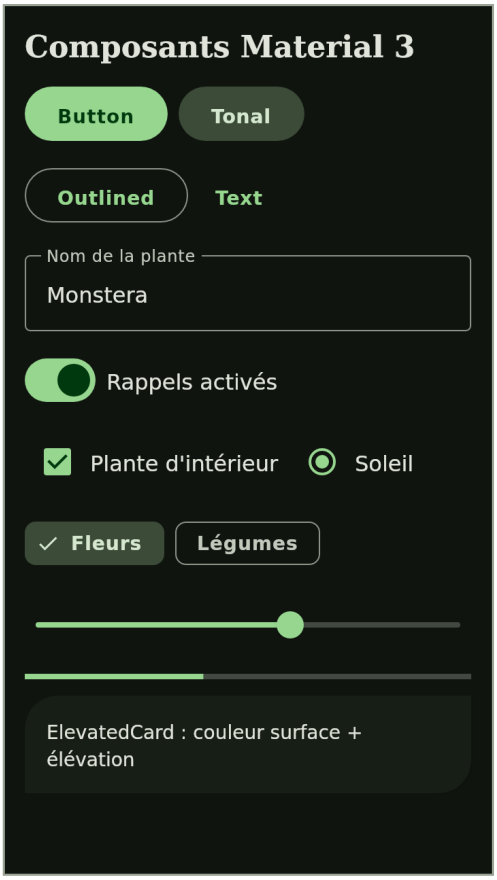
8. Utiliser le thème dans ses composables

8.1 Les composants Material s'adaptent tout seuls

Boutons, champs de texte, interrupteurs, cases à cocher, puces, curseurs… tous lisent le thème. Le même code donne ces deux rendus :



Composants — mode clair



Composants — mode sombre

8.2 Lire une valeur du thème

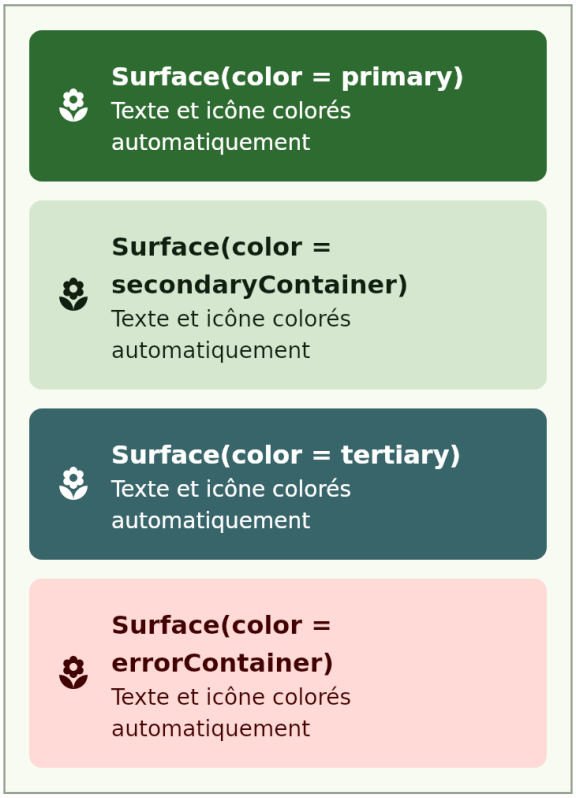
Exemple : la carte d'une plante

```
@Composable
fun CartePlante(p: Plante) {
    Card(shape = MaterialTheme.shapes.medium, modifier = Modifier.fillMaxWidth()) {
        Row(Modifier.padding(16.dp), verticalAlignment = Alignment.CenterVertically) {
            Icon(
                Icons.Filled.LocalFlorist, contentDescription = null,
                tint = MaterialTheme.colorScheme.primary,      // couleur du thème
            )
            Spacer(Modifier.width(12.dp))
            Column(Modifier.weight(1f)) {
                Text(p.nom, style = MaterialTheme.typography.titleMedium)
                Text(p.info, style = MaterialTheme.typography.bodyMedium)
            }
            FilledTonalButton(onClick = {}) { Text("Arroser") }
        }
    }
}
```

8.3 Surface et la couleur de contenu automatique

Le composable `Surface` fait plus que peindre un fond : il choisit aussi la **couleur de contenu** (`LocalContentColor`) correspondante. Si le fond est `primary`, les `Text` et `Icon` à l'intérieur deviennent automatiquement `onPrimary`.

```
Surface(color = MaterialTheme.colorScheme.primary, shape = MaterialTheme.shapes.small) {
    Row(Modifier.padding(16.dp)) {
        Icon(Icons.Filled.LocalFlorist, null)   // pas de couleur précisée...
        Text("Surface(color = primary)")         // ...et pourtant bien lisible !
    }
}
```



Le texte et l'icône prennent la couleur « on… » adaptée

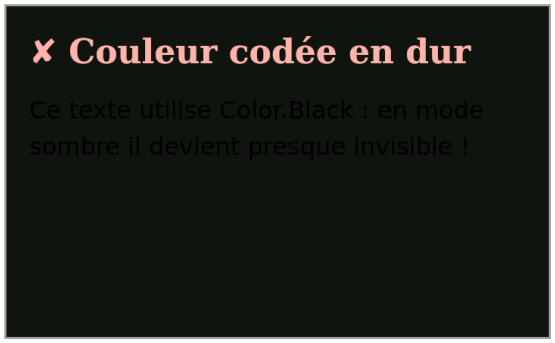
> **Astuce**
> Si vous voulez la couleur de contenu de n'importe quelle couleur du thème, utilisez `contentColorFor(MaterialTheme.colorScheme.tertiary)`.

8.4 L'erreur classique : la couleur codée en dur

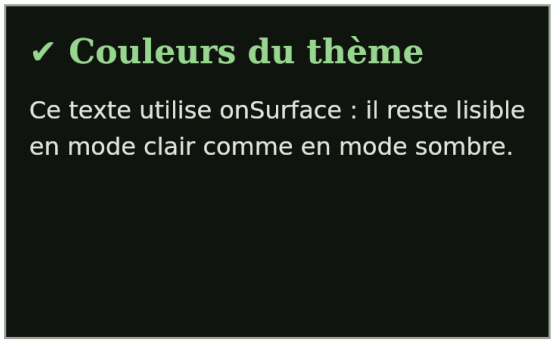
Écrire `color = Color.Black` fonctionne… jusqu'au jour où l'utilisateur passe en mode sombre :

```
// ✘ À éviter
Text("Mon texte", color = Color.Black)

// ✔ Correct
Text("Mon texte", color = MaterialTheme.colorScheme.onSurface)
// ou, encore plus simple, ne rien préciser : Text utilise LocalContentColor
Text("Mon texte")
```



✘ Color.Black en mode sombre : illisible



✔ onSurface : lisible

9. Aller plus loin

9.1 Ajouter ses propres couleurs (CompositionLocal)

`ColorScheme` n'a pas de rôle « succès » ou « avertissement ». Pour en ajouter, on crée une classe et on la diffuse avec un **CompositionLocal**, exactement comme le fait `MaterialTheme` en interne.

ui/theme/CouleursEtendues.kt

```
@Immutable
data class CouleursEtendues(
    val succes: Color, val surSucces: Color,
    val alerte: Color, val surAlerte: Color,
)

val CouleursClaires = CouleursEtendues(Color(0xFF1B6D2F), Color.White,
                                       Color(0xFF8A5100), Color.White)
val CouleursSombres = CouleursEtendues(Color(0xFF8BD89A), Color(0xFF00391A),
                                       Color(0xFFFFB870), Color(0xFF4A2800))

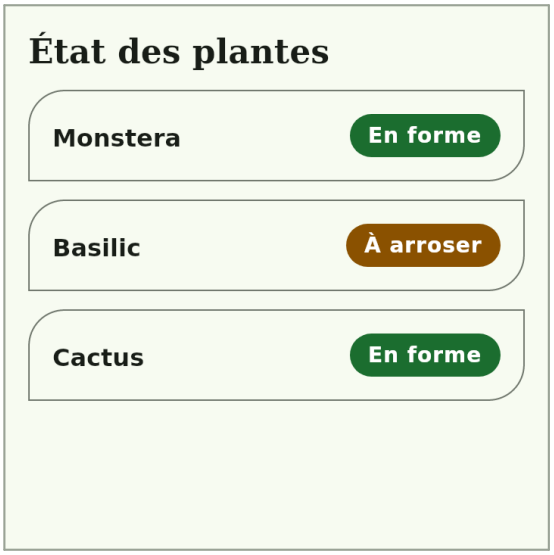
// 1) Déclarer le « canal » avec une valeur par défaut
val LocalCouleursEtendues = staticCompositionLocalOf { CouleursClaires }

// 2) Un accès pratique, comme MaterialTheme.colorScheme
object JardinThemeExt {
    val couleurs: CouleursEtendues
        @Composable @ReadOnlyComposable
        get() = LocalCouleursEtendues.current
}
```

Puis, dans `JardinTheme`, on fournit la bonne valeur selon le mode :

```
val etendues = if (darkTheme) CouleursSombres else CouleursClaires
CompositionLocalProvider(LocalCouleursEtendues provides etendues) {
    MaterialTheme(colorScheme = colorScheme, typography = JardinTypography,
                  shapes = JardinShapes, content = content)
}

// Utilisation dans un écran :
val ext = JardinThemeExt.couleurs
Surface(color = ext.succes, contentColor = ext.surSucces, shape = CircleShape) {
    Text("En forme", Modifier.padding(horizontal = 12.dp, vertical = 6.dp))
}
```



Badges « succès » / « alerte » — clair



Les mêmes badges — sombre

9.2 Thème imbriqué : changer le style d'une seule zone

On peut placer un `MaterialTheme` à l'intérieur d'un autre. Le thème intérieur ne s'applique qu'à sa zone. Avec `copy()`, on part du thème courant et on ne modifie que quelques rôles :

```
MaterialTheme(
    colorScheme = MaterialTheme.colorScheme.copy(
        primary = MaterialTheme.colorScheme.tertiary,
        onPrimary = MaterialTheme.colorScheme.onTertiary,
        surfaceVariant = MaterialTheme.colorScheme.tertiaryContainer,
    )
) {
    // Ici, Button et Card utilisent la couleur tertiaire
    Button(onClick = {}) { Text("Bouton premium") }
    CartePlante(Plante("Orchidée", "Arrosage : 1 fois / 10 jours"))
}
```



La zone « Premium » utilise un thème imbriqué

10. Bonnes pratiques et mémo

10.1 Bonnes pratiques

- Toujours envelopper l'application **et** les `@Preview` dans votre thème.
- Ne jamais coder une couleur en dur dans un écran : passer par `MaterialTheme.colorScheme`.
- Respecter les paires `X / onX` pour garantir le contraste.
- Utiliser les styles `MaterialTheme.typography.…` plutôt que des `fontSize` isolés.
- Toujours fournir une palette claire **et** une palette sombre.
- Tester vos écrans dans les deux modes (deux `@Preview`).
- Générer vos palettes avec Material Theme Builder pour des contrastes corrects.

10.2 Mémo

Je veux…	J'écris…
Lire une couleur	`MaterialTheme.colorScheme.primary`
Lire un style de texte	`MaterialTheme.typography.titleLarge`
Lire une forme	`MaterialTheme.shapes.medium`
Savoir si le téléphone est en sombre	`isSystemInDarkTheme()`
Couleurs du fond d'écran (Android 12+)	`dynamicLightColorScheme(context)`
La couleur de texte adaptée à un fond	`contentColorFor(couleur)`
Modifier quelques couleurs localement	`MaterialTheme(colorScheme = …copy(…)) { }`
Ajouter mes propres valeurs au thème	`staticCompositionLocalOf + CompositionLocalProvider`

10.3 Exercices

Exercice
1. Changez la couleur de départ de « Mon Jardin » pour un thème bleu « Océan ». Générez la palette avec Material Theme Builder et remplacez `Color.kt`.

Exercice
2. Ajoutez un interrupteur « Mode sombre » dans un écran de réglages qui force `darkTheme` (chapitre 5.3).

Exercice
3. Changez la forme `medium` pour `CutCornerShape(12.dp)` et observez les cartes.

Exercice
4. Ajoutez une couleur étendue « info » (bleu) à `CouleursEtendues` et affichez un badge « Nouveau ».

Exercice
5. Cherchez dans votre projet toutes les occurrences de `Color(0x` en dehors du dossier `ui/theme` et remplacez-les par des rôles du thème.

10.4 Contenu de l'archive

Dossier	Contenu
`cours-themes-jetpack-compose.pdf`	Ce cours.
`captures/`	Toutes les captures d'écran du cours, en haute résolution (PNG).
`code-android/`	Les fichiers Kotlin de l'exemple « Mon Jardin » à copier dans un projet Android Studio : Color.kt, Type.kt, Shape.kt, CouleursEtendues.kt, Theme.kt, EcranJardin.kt, MainActivity.kt.
`generateur-captures/`	Le petit projet Compose (Kotlin/JVM) qui a produit les captures : le même code de thème, rendu hors écran.

Note sur les captures : elles ont été générées par le vrai moteur de rendu Compose (Compose Multiplatform, bibliothèque Material 3) à partir du code présenté dans ce cours. Sur un téléphone Android, la police système (Roboto) et certains détails d'espacement peuvent légèrement différer.